

Diabetes Prediction

July 28, 2023

0.1 Description:

The objective of this diabetes dataset is to predict whether patient has diabetes or not. The datasets consists of several medical predictor(Independent) variables and one target variable, (Outcome). Predictor variables includes pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, DiabetesPedigreeFunction, age and Outcome.

Dataset url : <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

0.1.1 Name of the columns with their meanings

- 1) Pregnancies → Number of times pregnant
- 2) Glucose → Plasma glucose concentration a 2 hours in an oral glucose tolerance test
- 3) BloodPressure → Diastolic blood pressure(mm Hg)
- 4) SkinThickness → Triceps skin fold thickness (mm)
- 5) Insulin → 2-Hour serum insulin (mu U/ml)
- 6) BMI → Body mass index (weight in kg/(height in m)²)
- 7) DiabetesPedigreeFunction → Diabetes pedigree function
- 8) Age → Age (years)
- 9) Outcome → Class variable (0 or 1) 268 or 768 are 1 500 of 768 are 0

0.1.2 Step 1 : Importing libraries like Numpy, Pandas, Matplotlib, Seaborn and scikit learn(Sklearn)

```
[1]: # Numpy
import numpy as np

#Pandas
import pandas as pd

#matplotlib
import matplotlib.pyplot as plt

# Seaborn
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

0.1.3 Step2 : Load the Dataset

```
[2]: # Load pima indians dataset
df = pd.read_csv("diabetes.csv")
```

0.1.4 Step 3 : Exploratory Data Analysis

Exploratory Data Analysis (EDA) is a step in the Data Analysis Process, where a number of techniques are used to better understand the dataset being used.

In this step, we will perform the below operations:

- 3.1) Understanding Your Variables 3.1.1) Head of the dataset 3.1.2) The shape of the dataset
3.1.3) List types of all columns 3.1.4) Info of the dataset 3.1.5) Summary of the dataset
- 3.2) Data Cleaning 3.2.1) Check the DUPLICATES 3.2.2) Check the NULL values

3.1.1) Head of the Dataset This head(n) function returns the first n rows for the object based on position. It is useful for quickly testing if your object has the right type of data in it. By default it will show 5 rows.

```
[3]: # Display first five records of data
df.head()
```

```
[3]:   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0             6     148             72             35         0  33.6
1             1      85             66             29         0  26.6
2             8     183             64              0         0  23.3
3             1      89             66             23        94  28.1
4             0     137             40             35       168  43.1

      DiabetesPedigreeFunction  Age  Outcome
0                   0.627     50         1
1                   0.351     31         0
2                   0.672     32         1
3                   0.167     21         0
4                   2.288     33         1
```

```
[4]: # Display last five records of the data
df.tail()
```

```
[4]:   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
763           10     101             76             48       180  32.9
764            2     122             70             27         0  36.8
765            5     121             72             23       112  26.2
766            1     126             60              0         0  30.1
767            1      93             70             31         0  30.4

      DiabetesPedigreeFunction  Age  Outcome
763                   0.171     63         0
```

764	0.340	27	0
765	0.245	30	0
766	0.349	47	1
767	0.315	23	0

```
[5]: # Display randomly any number of records of data
df.sample(10)
```

```
[5]:      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
523             9      130             70             0         0  34.2
669             9      154             78             30        100  30.9
444             4      117             62             12         0  29.7
630             7      114             64             0         0  27.4
260             3      191             68             15        130  30.9
482             4       85             58             22         49  27.8
627             0      132             78             0         0  32.4
458            10      148             84             48        237  37.6
682             0       95             64             39        105  44.6
330             8      118             72             19         0  23.1

      DiabetesPedigreeFunction  Age  Outcome
523                      0.652   45         1
669                      0.164   45         0
444                      0.380   30         1
630                      0.732   34         1
260                      0.299   34         0
482                      0.306   28         0
627                      0.393   21         0
458                      1.001   51         1
682                      0.366   22         0
330                      1.476   46         0
```

3.1.2) The shape of the dataset This shape () function gives us the number of rows and columns of the dataset.

```
[6]: #Number of rows and columns
df.shape
```

```
[6]: (768, 9)
```

No.of Rows=768

No.of Columns=9

3.1.3) List types of all columns Using df.dtypes, we get the list of types of all the columns in our dataset.

```
[7]: #List the types of all columns.
df.dtypes
```

```
[7]: Pregnancies      int64
      Glucose          int64
      BloodPressure    int64
      SkinThickness    int64
      Insulin          int64
      BMI              float64
      DiabetesPedigreeFunction float64
      Age              int64
      Outcome          int64
      dtype: object
```

3.1.4) Info of the dataset info() is used to check the Information about the data and the datatypes of each respective attribute.

```
[8]: #finding out if the dataset contains any null value
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies            768 non-null   int64
1   Glucose                768 non-null   int64
2   BloodPressure          768 non-null   int64
3   SkinThickness          768 non-null   int64
4   Insulin                768 non-null   int64
5   BMI                    768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                    768 non-null   int64
8   Outcome                768 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

3.1.5) Summary of the dataset The describe() method is used for calculating some statistical data like percentile, mean and std of the numerical values of the Series or DataFrame. It analyzes both numeric and object series and also the DataFrame column sets of mixed data types.

```
[9]: # Statistical summary
df.describe()
```

```
[9]:      Pregnancies      Glucose  BloodPressure  SkinThickness      Insulin  \
count      768.000000    768.000000      768.000000      768.000000    768.000000
mean         3.845052    120.894531        69.105469        20.536458     79.799479
std          3.369578     31.972618        19.355807        15.952218    115.244002
```

min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

Observation: In the above table, the min value of columns 'Glucose','Blood Pressure', 'SkinThickness', 'Insulin', 'BMI' is zero (0). It is clear that these values can't be zero. So we impute the mean values of these respective columns instead of zero.

0.1.5 3.2) Data Cleaning

3.2.1) Drop the Duplicates check if there are any duplicate rows or not, if exist then we should remove them from the dataframe.

```
[10]: # check the shape before drop the duplicates
df.shape
```

```
[10]: (768, 9)
```

```
[11]: df=df.drop_duplicates()
```

```
[12]: # check the shape after drop the duplicates
df.shape
```

```
[12]: (768, 9)
```

Before dropping and after dropping the duplicates the data set has same shape so no duplicates are there in the dataset.

0.1.6 3.2.2) Check the NULL Values

Using `isnull.sum()` function we can see the null values present in every column in the dataset.

```
[13]: # Count of null, values

# check the missing values in any column
# Display number of null values in every column in dataset
df.isnull().sum()
```

```
[13]: Pregnancies      0
      Glucose          0
      BloodPressure    0
      SkinThickness    0
      Insulin          0
      BMI              0
      DiabetesPedigreeFunction  0
      Age              0
      Outcome          0
      dtype: int64
```

There is no NULL values in the given dataset.

```
[14]: df.columns
```

```
[14]: Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
          'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
          dtype='object')
```

Check the no.of Zero values in dataset

```
[15]: print('No. of zero values in Glucose',df[df['Glucose']==0].shape[0])
```

No. of zero values in Glucose 5

```
[16]: print('No. of zero values in Blood Pressure',df[df['BloodPressure']==0].
      ↪shape[0])
```

No. of zero values in Blood Pressure 35

```
[17]: print('No. of zero values in SkinThickness' ,df[df['SkinThickness']==0].
      ↪shape[0])
```

No. of zero values in SkinThickness 227

```
[18]: print('No. of zero values in Insulin' ,df[df['Insulin']==0].shape[0])
```

No. of zero values in Insulin 374

```
[19]: print('No. of zero values in BMI',df[df['BMI']==0].shape[0])
```

No. of zero values in BMI 11

Replace no.of zero values with mean of that columns

```
[20]: df['Glucose']=df['Glucose'].replace(0, df['Glucose'].mean())
      print('No. of zero values in Glucose' ,df[df['Glucose']==0].shape[0])
```

No. of zero values in Glucose 0

```
[21]: df['BloodPressure']=df['BloodPressure'].replace(0, df['BloodPressure'].mean())
      df['SkinThickness']=df['SkinThickness'].replace(0,df['SkinThickness'].mean())
```

```
df['Insulin']=df['Insulin'].replace(0,df['Insulin'].mean())
df['BMI']=df['BMI'].replace(0, df['BMI'].mean())
```

```
[22]: df.describe()
```

```
[22]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.681605	72.254807	26.606479	118.660163
std	3.369578	30.436016	12.115932	9.631241	93.080358
min	0.000000	44.000000	24.000000	7.000000	14.000000
25%	1.000000	99.750000	64.000000	20.536458	79.799479
50%	3.000000	117.000000	72.000000	23.000000	79.799479
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	32.450805	0.471876	33.240885	0.348958
std	6.875374	0.331329	11.760232	0.476951
min	18.200000	0.078000	21.000000	0.000000
25%	27.500000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

0.2 4) Data Visualization

0.2.1 4.1) Count plot

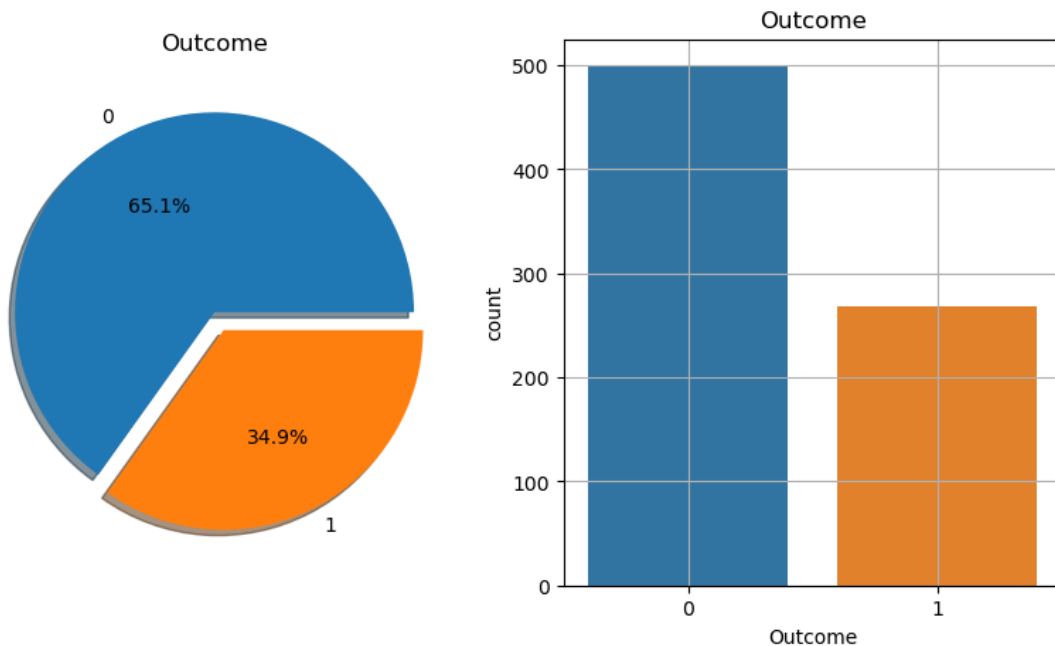
```
[23]: # outcome count plot

f,ax=plt.subplots(1,2,figsize=(10,5))
df['Outcome'].value_counts().plot.pie(explode=[0,0.1], autopct='%1.1f%%',
    ↪ax=ax[0], shadow=True)
ax[0].set_title('Outcome')
ax[0].set_ylabel('')
sns.countplot('Outcome',data=df,ax=ax[1])
ax[1].set_title('Outcome')
N,P=df['Outcome'].value_counts()
print('Negative (0):',N)
print('Positive (1):',P)

plt.grid()
plt.show()
```

Negative (0): 500

Positive (1): 268



Out of total 768 people, 268 are diabetic (Positive (1)) and 500 are non-diabetic (negative(0)).

In the outcome column, 1 represents diabetes positive and 0 represents diabetes negative.

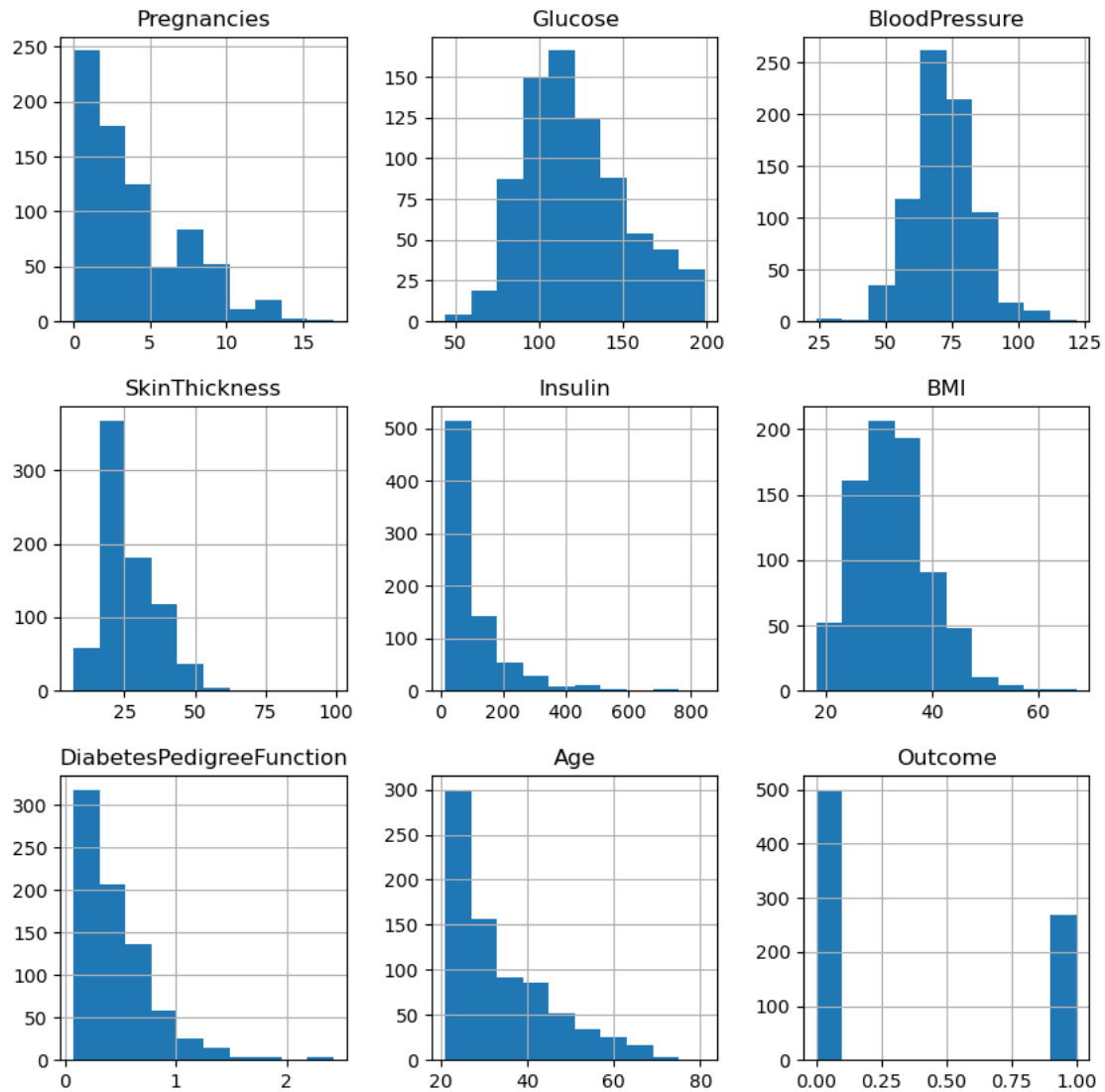
The countplot tells us that the dataset is Imbalanced, as number of patients who don't have diabetes is more than those who have diabetes.

0.2.2 4.2) Histograms

Histograms are one of the most common graphs used to display numeric data.

distribution of the data - Whether the data is normally distributed or if it's skewed (to the left or right)

```
[24]: # Histogram of each feature
df.hist(bins=10,figsize=(10,10))
plt.show()
```

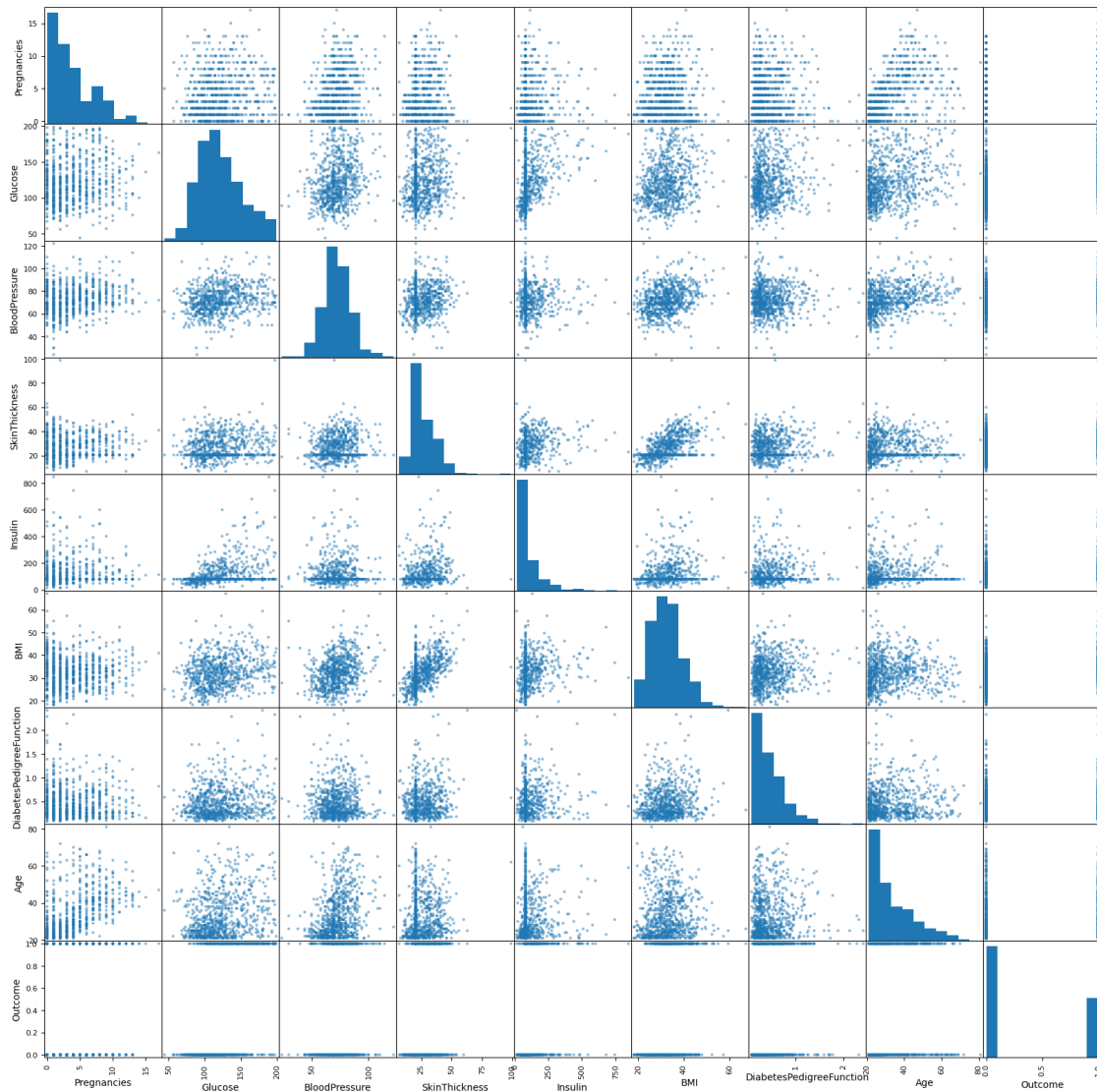



0.2.3 4.3) Scatter plot

Scatterplots are useful for many reasons: like correlation matrices, it allows us to quickly understand a relationship between two variables.

```
[25]: # Scatter plot matrix

from pandas.plotting import scatter_matrix
scatter_matrix(df, figsize = (20, 20));
```

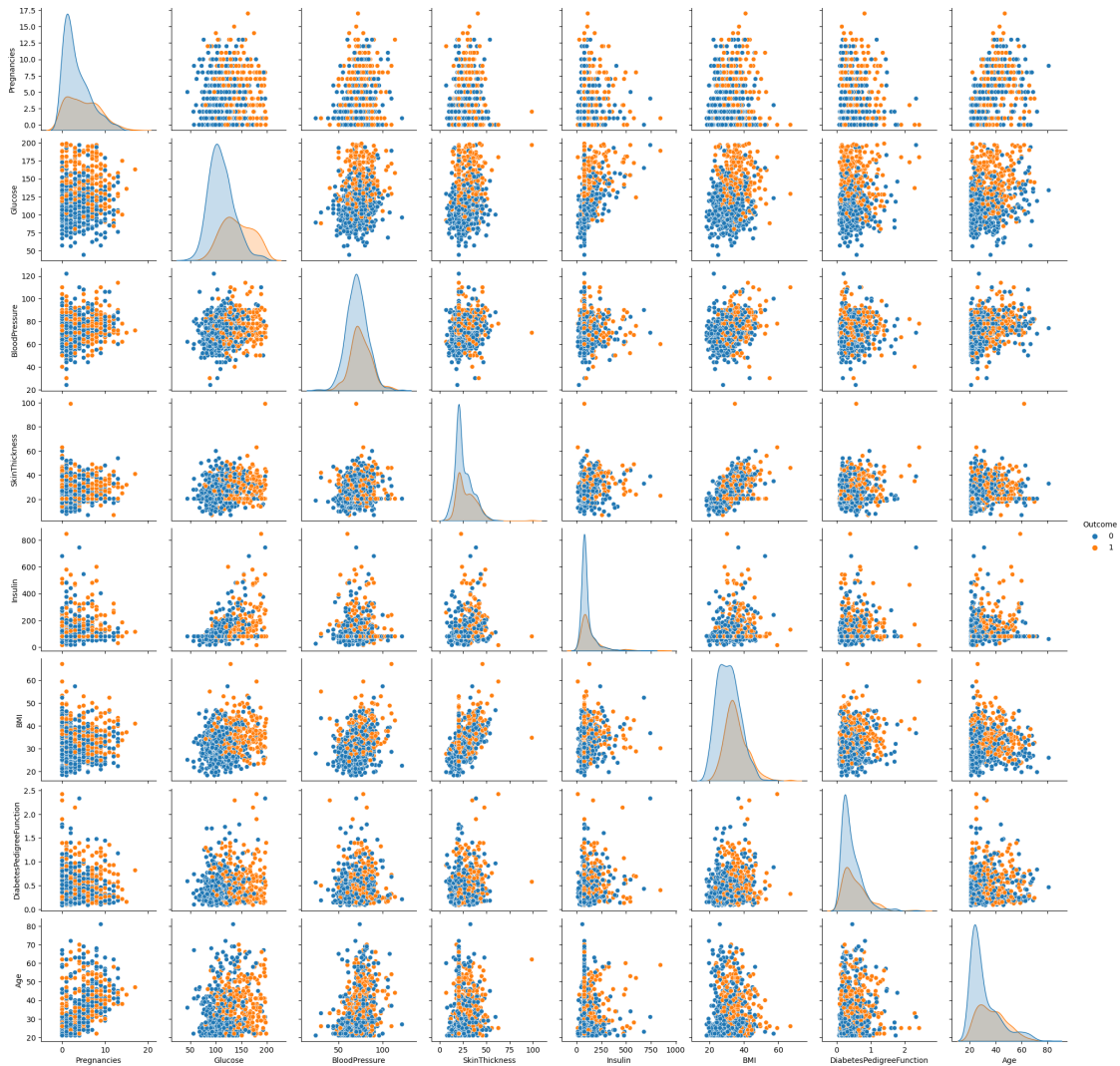


0.2.4 4.4) Pairplot

`sns.pairplot()` is a great way to create scatterplots between all of our variables.

```
[26]: # Pairplot

sns.pairplot(data = df, hue = 'Outcome')
plt.show()
```



0.2.5 4.5) Analyzing relationships between variables

Correlation analysis Correlation analysis in data science is a statistical technique used to measure the strength and direction of the relationship between two or more variables in a dataset. It helps data scientists understand how changes in one variable are associated with changes in another. By calculating correlation coefficients, such as Pearson's correlation coefficient for continuous variables or rank-based correlations for non-linear or ordinal data, data scientists can identify patterns and dependencies in the data. This analysis is valuable for feature selection, identifying potential predictor variables, and gaining insights into the interactions between different aspects of the dataset, facilitating better decision-making and predictive modeling.

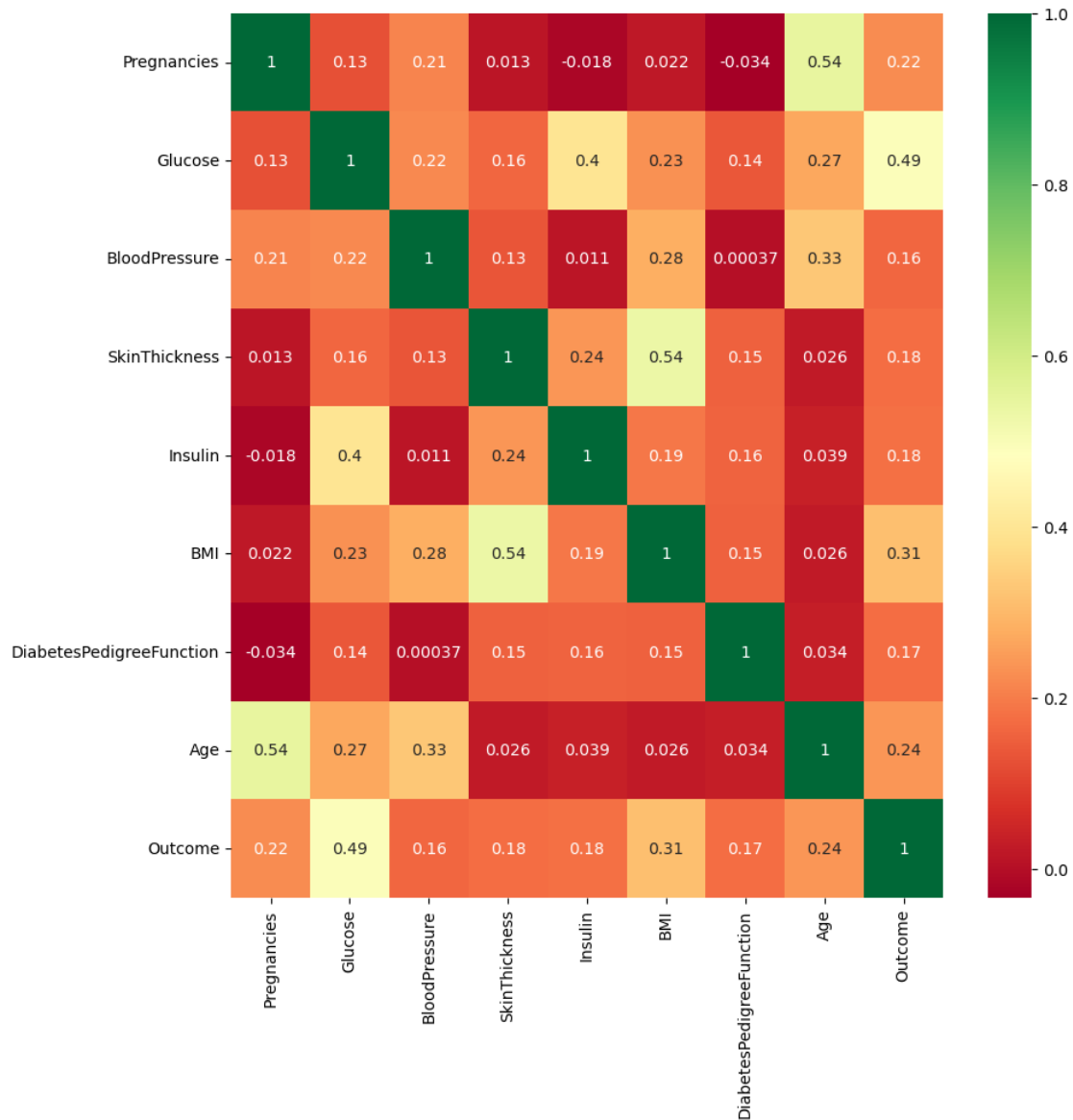
```
[27]: import seaborn as sns

#get correlations of each features in dataset
```

```

corrmat = df.corr()
top_corr_features = corrmat.index
plt.figure(figsize=(10,10))
#plot heat map
g=sns.heatmap(df[top_corr_features].corr(),annot=True,cmap="RdYlGn")

```



Observations: From the correaltion heatmap, we can see that there is a high correlation between Outcome and (Pregnancies, Glucose, BMI, Age, Insulin). We can select these features to accept input from the user and predict the outcome.

0.3 5) Split the data frame into X & y

```
[28]: target_name = 'Outcome'

# Separate object for target feature
y=df[target_name]

#Separate Object for Input Features
X = df.drop(target_name, axis=1)
```

```
[29]: X.head()
```

```
[29]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148.0	72.0	35.000000	79.799479	33.6	
1	1	85.0	66.0	29.000000	79.799479	26.6	
2	8	183.0	64.0	20.536458	79.799479	23.3	
3	1	89.0	66.0	23.000000	94.000000	28.1	
4	0	137.0	40.0	35.000000	168.000000	43.1	

	DiabetesPedigreeFunction	Age
0	0.627	50
1	0.351	31
2	0.672	32
3	0.167	21
4	2.288	33

```
[30]: y.head()
```

```
[30]:
```

0	1
1	0
2	1
3	0
4	1

Name: Outcome, dtype: int64

0.4 6) Apply Feature Scalling

```
[31]: # Apply Standard Scaler
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
SSX = scaler.transform(X)
```

Inplace of Standard Scaler you have to try other data scaling technique like normalizer, minmax scaler and binarizer

0.5 7) Train Test Split

```
[32]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(SSX, y, test_size=0.2,
↳ random_state=7)
```

```
[33]: X_train.shape, y_train.shape
```

```
[33]: ((614, 8), (614,))
```

```
[34]: X_test.shape, y_test.shape
```

```
[34]: ((154, 8), (154,))
```

0.6 8) Build the Classification Algorithms

0.6.1 8.1) Logistic Regression

```
[35]: from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(solver='liblinear', multi_class='ovr')
lr.fit(X_train, y_train)
```

```
[35]: LogisticRegression(multi_class='ovr', solver='liblinear')
```

0.6.2 8.2) KNeighborsClassifier(KNN)

```
[36]: from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
```

```
[36]: KNeighborsClassifier()
```

0.6.3 8.3) Naive-Bayes Classifier

```
[37]: from sklearn.naive_bayes import GaussianNB
nb = GaussianNB()
nb.fit(X_train, y_train)
```

```
[37]: GaussianNB()
```

0.6.4 8.4) Support Vector Machine

```
[38]: from sklearn.svm import SVC
sv = SVC()
sv.fit(X_train, y_train)
```

```
[38]: SVC()
```

0.6.5 8.5) Decision Tree

```
[39]: from sklearn.tree import DecisionTreeClassifier  
      dt = DecisionTreeClassifier()  
      dt.fit(X_train, y_train)
```

```
[39]: DecisionTreeClassifier()
```

0.6.6 8.6) Random Forest

```
[40]: from sklearn.ensemble import RandomForestClassifier  
      rf = RandomForestClassifier()  
      rf.fit(X_train, y_train)
```

```
[40]: RandomForestClassifier()
```

0.7 9) Making Prediction

0.7.1 9.1) Making Prediction on test by using Logistic Regression

```
[41]: # display the shape of test data  
      X_test.shape
```

```
[41]: (154, 8)
```

```
[42]: # making prediction on test dataset  
      lr_pred=lr.predict(X_test)
```

```
[43]: # display the shape of predict data  
      lr_pred.shape
```

```
[43]: (154,)
```

0.7.2 9.2) Making Prediction on test by using KNN

```
[44]: # making prediction on test dataset  
      knn_pred=knn.predict(X_test)
```

0.7.3 9.3) Making Prediction on test by using Naive-Bayes

```
[45]: # making prediction on test dataset  
      nb_pred=nb.predict(X_test)
```

0.7.4 9.4) Making Prediction on test by using SVM

```
[46]: # making prediction on test dataset  
      sv_pred=sv.predict(X_test)
```

0.7.5 9.5) Making Prediction on test by using Decision Tree

```
[47]: # making prediction on test dataset
dt_pred=dt.predict(X_test)
```

0.7.6 9.6) Making Prediction on test by using Random Forest

```
[48]: # making prediction on test dataset
rf_pred=rf.predict(X_test)
```

0.8 10) Model Evaluation

0.8.1 10.1) Train and Test Score

```
[49]: # Train and Test Score of Logistic Regression
from sklearn.metrics import accuracy_score
print("Train Accuracy score of Logistic Regression",lr.
      ↪score(X_train,y_train)*100)
print("Test Accuracy score of Logistic Regression",lr.score(X_test,y_test)*100)
print("Accuracy of score Logistic_
      ↪Regression",accuracy_score(y_test,lr_pred)*100)
```

Train Accuracy score of Logistic Regression 77.36156351791531

Test Accuracy score of Logistic Regression 77.27272727272727

Accuracy of score Logistic Regression 77.27272727272727

```
[50]: # Train and Test Score of KNN
print("Train Accuracy score of KNN",knn.score(X_train,y_train)*100)
print("Test Accuracy score of KNN",knn.score(X_test,y_test)*100)
print("Accuracy of score KNN",accuracy_score(y_test,knn_pred)*100)
```

Train Accuracy score of KNN 81.10749185667753

Test Accuracy score of KNN 74.67532467532467

Accuracy of score KNN 74.67532467532467

```
[51]: # Train and Test Score of Naive-Bayes
print("Train Accuracy score of Naive-Bayes",nb.score(X_train,y_train)*100)
print("Test Accuracy score of Naive-Bayes",nb.score(X_test,y_test)*100)
print("Accuracy of score Naive-Bayes",accuracy_score(y_test,nb_pred)*100)
```

Train Accuracy score of Naive-Bayes 74.2671009771987

Test Accuracy score of Naive-Bayes 74.02597402597402

Accuracy of score Naive-Bayes 74.02597402597402

```
[52]: # Train and Test Score of SVM
print("Train Accuracy score of SVM",sv.score(X_train,y_train)*100)
print("Test Accuracy score of SVM",sv.score(X_test,y_test)*100)
print("Accuracy of score SVM",accuracy_score(y_test,sv_pred)*100)
```


Train Accuracy score of SVM 81.92182410423453
Test Accuracy score of SVM 83.11688311688312
Accuracy of score SVM 83.11688311688312

```
[53]: # Train and Test Score of Decision Tree
print("Train Accuracy score of Decision Tree",dt.score(X_train,y_train)*100)
print("Test Accuracy score of Decision Tree",dt.score(X_test,y_test)*100)
print("Accuracy of score Decision Tree",accuracy_score(y_test,dt_pred)*100)
```

Train Accuracy score of Decision Tree 100.0
Test Accuracy score of Decision Tree 81.16883116883116
Accuracy of score Decision Tree 81.16883116883116

```
[54]: # Train and Test Score of Random Forest
print("Train Accuracy score of Random Forest",rf.score(X_train,y_train)*100)
print("Test Accuracy score of Random Forest",rf.score(X_test,y_test)*100)
print("Accuracy of score Random Forest",accuracy_score(y_test,rf_pred)*100)
```

Train Accuracy score of Random Forest 100.0
Test Accuracy score of Random Forest 78.57142857142857
Accuracy of score Random Forest 78.57142857142857

0.9 10.2) Confusion Matrix

Confusion matrices is a Table which is used to describe the performance of classification problem

It visualizes the accuracy of a classifier by comparing predicted values with actual values.

The terms used in confusion matrices are True Positive(TP), True Negative(TN), False Positive(FP) and False Negative(FN)

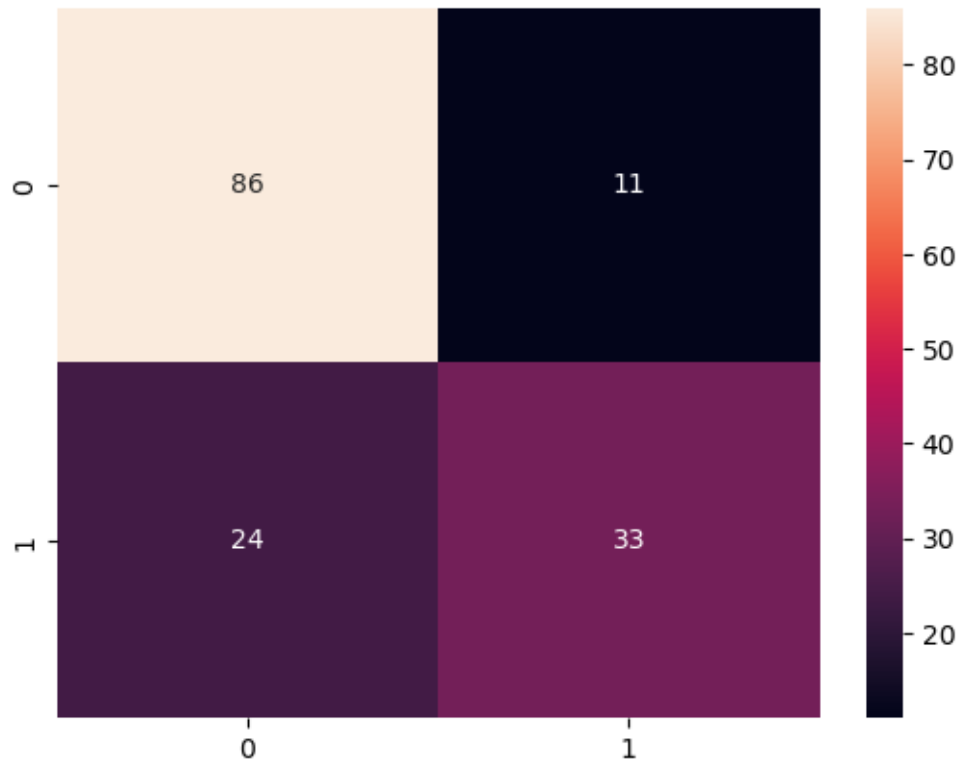
0.9.1 10.2.1) Confusion Matrix of Logistic Regression

```
[55]: from sklearn.metrics import classification_report, confusion_matrix
# confusion Matrix of Logistic Regression
cm=confusion_matrix(y_test, lr_pred)
cm
```

```
[55]: array([[86, 11],
        [24, 33]], dtype=int64)
```

```
[56]: sns.heatmap(confusion_matrix(y_test, lr_pred), annot=True, fmt="d")
```

```
[56]: <AxesSubplot:>
```



```
[57]: TN = cm[0,0]

FP = cm[0,1]

FN = cm[1,0]

TP = cm[1,1]

TN, FP, FN, TP
```

```
[57]: (86, 11, 24, 33)
```

```
[58]: # Making the Confusion Matrix Of Logistic Regression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
cm=confusion_matrix(y_test, lr_pred)

print('TN - True Negative {}'.format(cm[0,0]))
print('FP False Positive {}'.format(cm[0,1]))
print('FN False Negative {}'.format(cm[1,0]))
print('TP True Positive {}'.format(cm[1,1]))
```

```

print('Accuracy Rate: {}'.format(np.divide(np.sum([cm[0,0],cm[1,1]]),np.
↪sum(cm))*100))
print('Misclassification Rate: {}'.format(np.divide(np.sum([cm[0,1],cm[1,0]]),
↪np.sum(cm))*100))

```

TN - True Negative 86
 FP False Positive 11
 FN False Negative 24
 TP True Positive 33
 Accuracy Rate: 77.27272727272727
 Misclassification Rate: 22.727272727272727

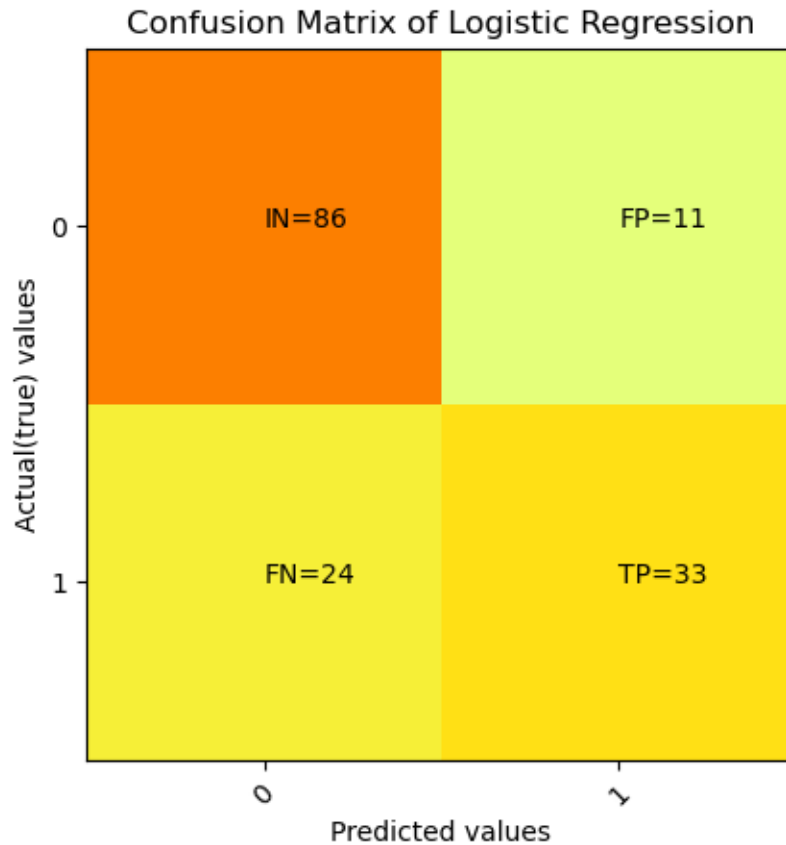
```

[59]: import matplotlib.pyplot as plt
plt.clf()
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Wistia)
classNames = ['0', '1']
plt.title('Confusion Matrix of Logistic Regression')
plt.ylabel('Actual(true) values')
plt.xlabel('Predicted values')
tick_marks = np.arange(len(classNames))
plt.xticks(tick_marks, classNames, rotation=45)
plt.yticks(tick_marks, classNames)
s = [['IN', 'FP'], ['FN', 'TP']]

for i in range(2):
    for j in range(2):
        plt.text(j,i, str(s[i][j])+"="+str(cm[i][j]))

plt.show()

```



```
[60]: pd.crosstab(y_test, lr_pred, margins=False)
```

```
[60]: col_0    0    1
Outcome
0         86   11
1         24   33
```

```
[61]: pd.crosstab(y_test, lr_pred, margins=True)
```

```
[61]: col_0    0    1  All
Outcome
0         86   11   97
1         24   33   57
All        110   44  154
```

```
[62]: pd.crosstab(y_test, lr_pred, rownames=['Actual values'], colnames=['Predicted_
      ↪values'], margins=True)
```

```
[62]: Predicted values    0    1  All
Actual values
```

0	86	11	97
1	24	33	57
All	110	44	154

0.10 PRECISION (PPV-Positive Predictive value)

It is the ratio of correctly predicted positive (TP) observations to the total predicted positive (TP+FP) observations.

Precision=TP/(TP+FP)

Where TP=True Positive

FP=False Positive

```
[63]: TP,FP
```

```
[63]: (33, 11)
```

```
[64]: Precision=TP/(TP+FP)
Precision
```

```
[64]: 0.75
```

```
[65]: # print precision score

precision_Score = TP / float (TP + FP)*100
print('Precision score: {0:0.4f}'.format(precision_Score))
```

Precision score: 75.0000

```
[66]: from sklearn.metrics import precision_score
print("precision Score is:", precision_score(y_test,lr_pred)*100)
print("Mircro Average precision Score is:", precision_score(y_test, lr_pred,␣
    ↪average='micro')*100)
print("Marcro Average precision Score is:", precision_score(y_test, lr_pred,␣
    ↪average='macro')*100)
print("Weighted Average precision Score is:", precision_score (y_test, lr_pred,␣
    ↪average='weighted')*100)
print("precision Score on Non weighted score is:", precision_score (y_test,␣
    ↪lr_pred, average=None)*100)
```

precision Score is: 75.0

Mircro Average precision Score is: 77.272727272727

Marcro Average precision Score is: 76.5909090909091

Weighted Average precision Score is: 77.00413223140497

precision Score on Non weighted score is: [78.18181818 75.]

```
[67]: print('Classification Report of Logistic Regression: \n',␣
    ↪classification_report(y_test, lr_pred, digits=4))
```

```

Classification Report of Logistic Regression:
      precision    recall  f1-score   support

     0       0.7818      0.8866      0.8309        97
     1       0.7500      0.5789      0.6535        57

 accuracy          0.7727          154
 macro avg       0.7659      0.7328      0.7422          154
 weighted avg    0.7700      0.7727      0.7652          154

```

0.11 Recall (True Positive Rate (TPR))

It is ratio of correctly predicted positive(Tp) observations to the total observations which are actually true.

```
[68]: recall_score = TP / float (TP + FN)*100
      print('recall score', recall_score)
```

recall score 57.89473684210527

```
[69]: from sklearn.metrics import recall_score
      print('Recall or Sensitivity score :',recall_score (y_test, lr_pred)*100)
```

Recall or Sensitivity score : 57.89473684210527

```
[70]: print("Mircro Average Recall Score is:", recall_score(y_test, lr_pred,
      ↪average='micro')*100)
      print("Marcro Average Recall Score is:", recall_score (y_test, lr_pred,
      ↪average='macro')*100)
      print("Weighted Average Recall Score is: ", recall_score (y_test, lr_pred,
      ↪average='weighted')*100)
      print("Recall Score on Non weighted score is:", recall_score (y_test, lr_pred,
      ↪average=None)*100)
```

Mircro Average Recall Score is: 77.27272727272727

Marcro Average Recall Score is: 73.27726532826912

Weighted Average Recall Score is: 77.27272727272727

Recall Score on Non weighted score is: [88.65979381 57.89473684]

```
[71]: print('Classification Report of Logistic Regression: \n',
      ↪classification_report(y_test, lr_pred, digits=4))
```

```

Classification Report of Logistic Regression:
      precision    recall  f1-score   support

     0       0.7818      0.8866      0.8309        97
     1       0.7500      0.5789      0.6535        57

 accuracy          0.7727          154

```

macro avg	0.7659	0.7328	0.7422	154
weighted avg	0.7700	0.7727	0.7652	154

0.12 False Positive Rate (FPR)

```
[72]: FPR = FP / float (FP+ TN)*100
print('False Positive Rate: {0:0.4f}'.format(FPR))
```

False Positive Rate: 11.3402

0.13 Specificity

```
[73]: specificity = TP / float (TN+ FP)*100
print('Specificity : {0:0.4f}'.format(specificity))
```

Specificity : 34.0206

0.14 F1 Score

```
[74]: from sklearn.metrics import f1_score
print('f1_score of macro : ',f1_score(y_test, lr_pred)*100)
```

f1_score of macro : 65.34653465346535

```
[75]: print('Mircro Average F1 Score is:', f1_score (y_test, lr_pred,
↪average='micro')*100)
print('Marcro Average F1 Score is:', f1_score (y_test, lr_pred,
↪average='macro')*100)
print('Weighted Average F1 Score is:', f1_score(y_test, lr_pred,
↪average='weighted')*100)
print('F1 Score on Non weighted score is:', f1_score (y_test, lr_pred,
↪average=None)*100)
```

Mircro Average F1 Score is: 77.27272727272727

Marcro Average F1 Score is: 74.21916104653944

Weighted Average F1 Score is: 76.52373933045479

F1 Score on Non weighted score is: [83.09178744 65.34653465]

0.15 Classification Report of Logistic Regression

```
[76]: from sklearn.metrics import classification_report
print('Classification Report of Logistic Regression: \n', classification_report,
↪(y_test, lr_pred, digits=4))
```

Classification Report of Logistic Regression:

	precision	recall	f1-score	support
0	0.7818	0.8866	0.8309	97

	1	0.7500	0.5789	0.6535	57
accuracy				0.7727	154
macro avg		0.7659	0.7328	0.7422	154
weighted avg		0.7700	0.7727	0.7652	154

0.16 ROC Curve & ROC AUC

ROC curve is one the important evaluating metrics that should be used to check the performance of an classification model. It is also called relative operating characteristic curve, because it is a comparison of two main characteristics (TPR and FPR). It is plotted between sensitivity (aka recall aka True Positive Rate) and False Positive Rate ($FPR = 1 - \text{specificity}$).

ROC (Receiver Operating Characteristic) Curve tells us about how good the model can distinguish between two things (e.g If a patient has a disease or no).

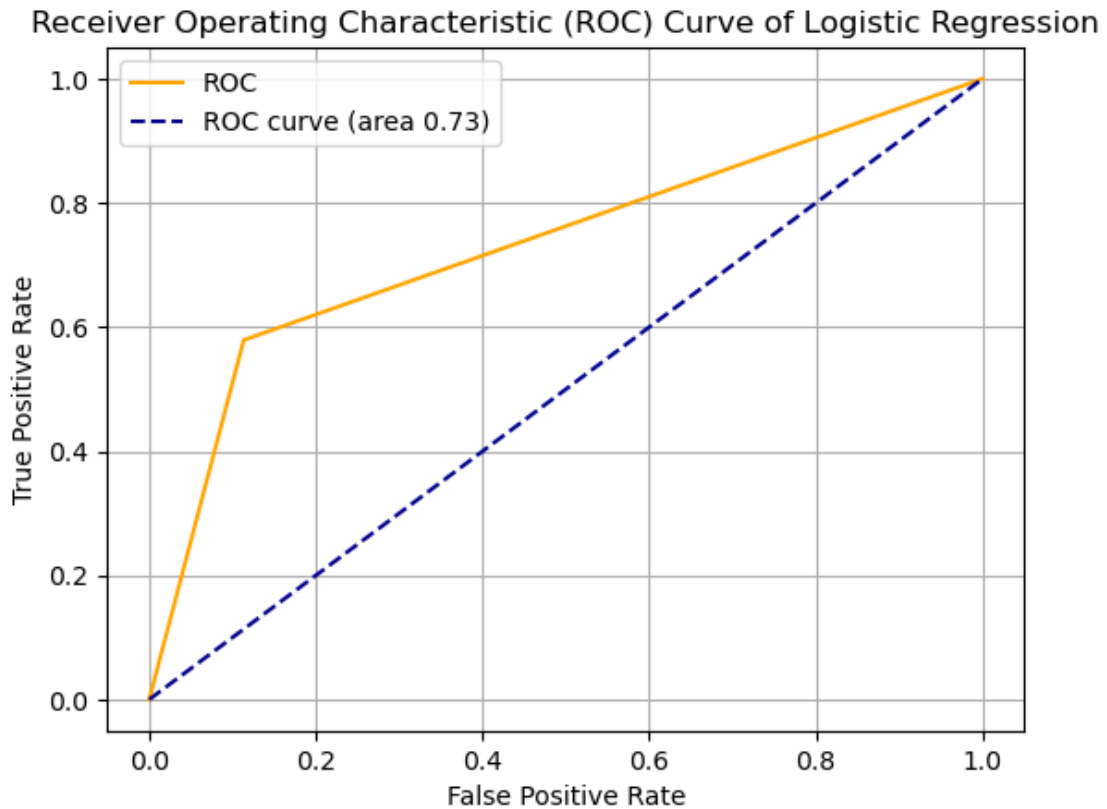
Area Under Curve (AUC) helps us to choose the best model amongst the models for which we have plotted the ROC curves

```
[77]: # Area Under Curve

auc = roc_auc_score(y_test, lr_pred)
print("ROC AUC SCORE of Logistic Regression is", auc)
```

ROC AUC SCORE of Logistic Regression is 0.7327726532826913

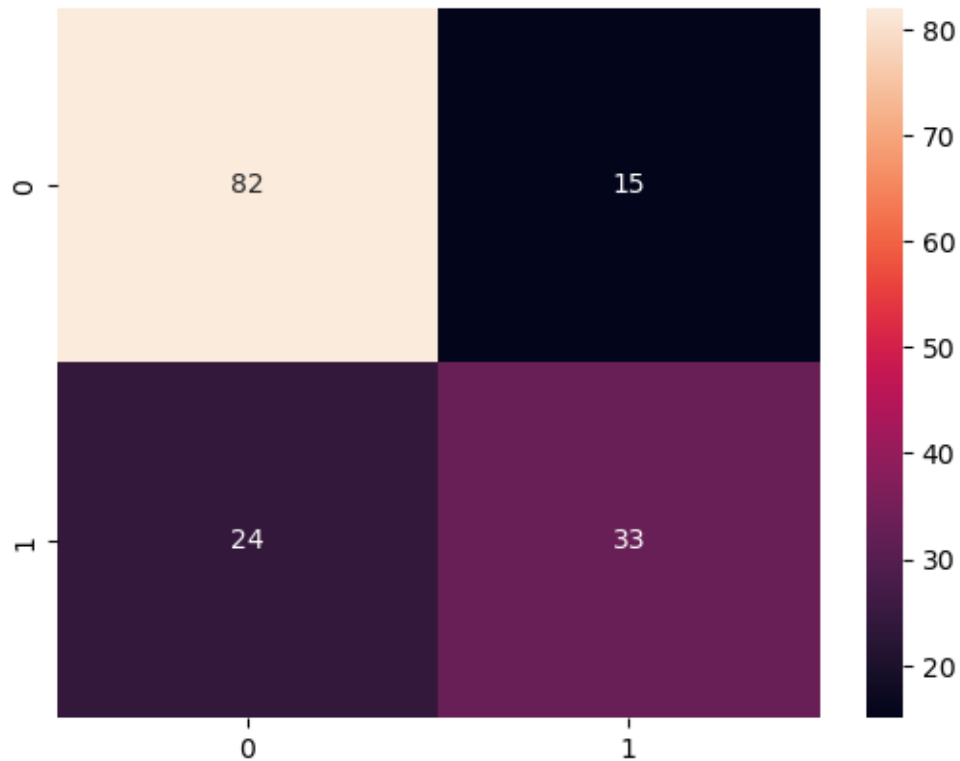
```
[78]: fpr, tpr, thresholds = roc_curve (y_test, lr_pred)
plt.plot(fpr, tpr, color='orange', label='ROC')
plt.plot([0, 1], [0, 1], color='darkblue', linestyle='--', label='ROC curve_
↳(area %0.2f)' % auc)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve of Logistic_
↳Regression')
plt.legend()
plt.grid()
plt.show()
```

0.17 10.2.2 Confusion Matrix of “KNN”

```
[79]: sns.heatmap(confusion_matrix(y_test,knn_pred),annot=True,fmt="d")
```

```
[79]: <AxesSubplot:>
```



```
[80]: # making the confusion matrix of knn
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
cm = confusion_matrix(y_test, knn_pred)

print('TN - True Negative {}'.format(cm[0,0]))
print('FP - False Positive {}'.format(cm[0,1]))
print('FN - False Negative {}'.format(cm[1,0]))
print('TP - True Positive {}'.format(cm[1,1]))
print('Accuracy Rate: {}'.format(np.divide(np.sum([cm[0,0], cm[1,1]]), np.
    ↳sum(cm))*100))
print('Misclassification Rate: {}'.format(np.divide(np.
    ↳sum([cm[0,1], cm[1,0]]), np.sum(cm))*100))
```

```
TN - True Negative 82
FP - False Positive 15
FN - False Negative 24
TP - True Positive 33
Accuracy Rate: 74.67532467532467
Misclassification Rate: 25.324675324675322
```

```
[81]: 75.97402597402598+24.025974025974026
```

```
[81]: 100.0
```

```
[82]: # classification report of knn
print(' Classification Report of KNN: \n',
      ↪classification_report(y_test,knn_pred,digits=4))
```

```
Classification Report of KNN:
              precision    recall  f1-score   support

     0       0.7736      0.8454      0.8079        97
     1       0.6875      0.5789      0.6286        57

 accuracy          0.7468        154
 macro avg       0.7305      0.7122      0.7182        154
 weighted avg    0.7417      0.7468      0.7415        154
```

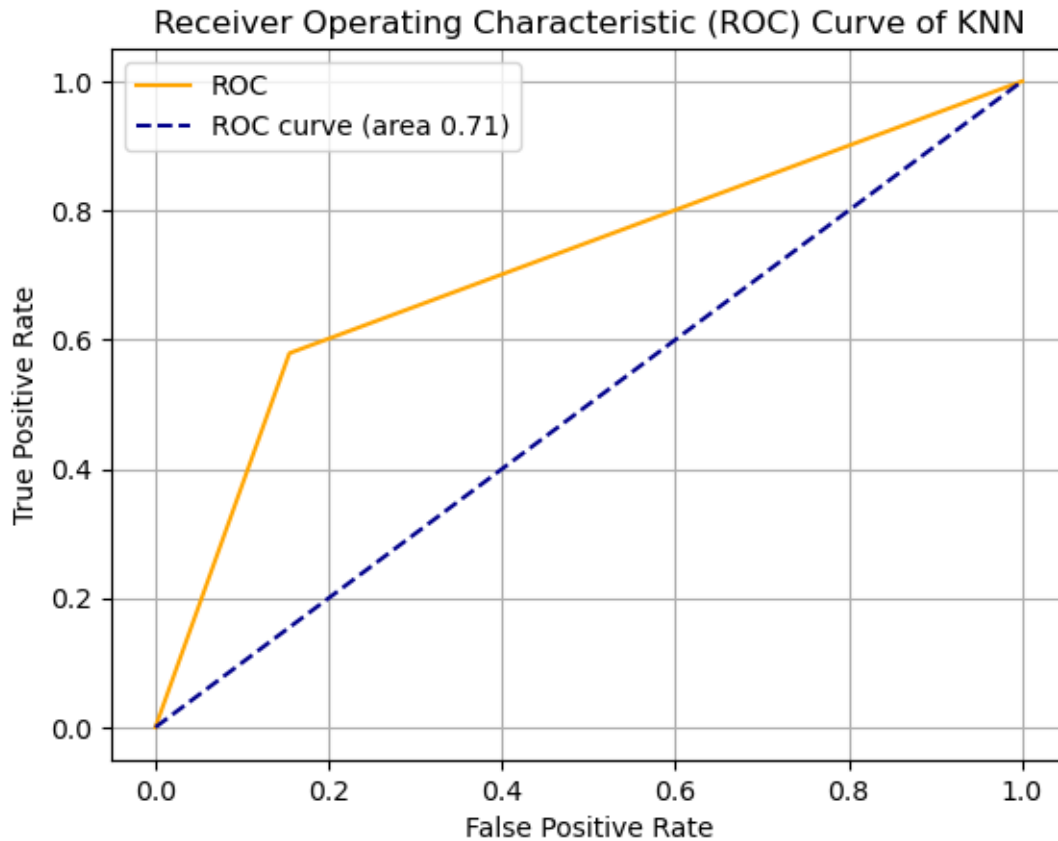
0.18 Area Under Curve of KNN

```
[83]: # Area Under Curve

auc = roc_auc_score(y_test, knn_pred)
print("ROC AUC SCORE of KNN is", auc)
```

ROC AUC SCORE of KNN is 0.7121540965816603

```
[84]: fpr, tpr, thresholds = roc_curve (y_test, knn_pred)
plt.plot(fpr, tpr, color='orange', label='ROC')
plt.plot([0, 1], [0, 1], color='darkblue', linestyle='--', label='ROC curve_
      ↪(area %0.2f)' % auc)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve of KNN')
plt.legend()
plt.grid()
plt.show()
```



0.18.1 10.2.3 Confusion Matrix of “Naive Bayes”

```
[85]: # making the confusion matrix of Naive Bayes
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
cm = confusion_matrix(y_test, nb_pred)

print('TN - True Negative {}'.format(cm[0,0]))
print('FP - False Positive {}'.format(cm[0,1]))
print('FN - False Negative {}'.format(cm[1,0]))
print('TP - True Positive {}'.format(cm[1,1]))
print('Accuracy Rate: {}'.format(np.divide(np.sum([cm[0,0], cm[1,1]]), np.
    ↳sum(cm))*100))
print('Misclassification Rate of Naive Bayes: {}'.format(np.divide(np.
    ↳sum([cm[0,1], cm[1,0]]), np.sum(cm))*100))
```

TN - True Negative 78
 FP - False Positive 19
 FN - False Negative 21
 TP - True Positive 36

Accuracy Rate: 74.02597402597402

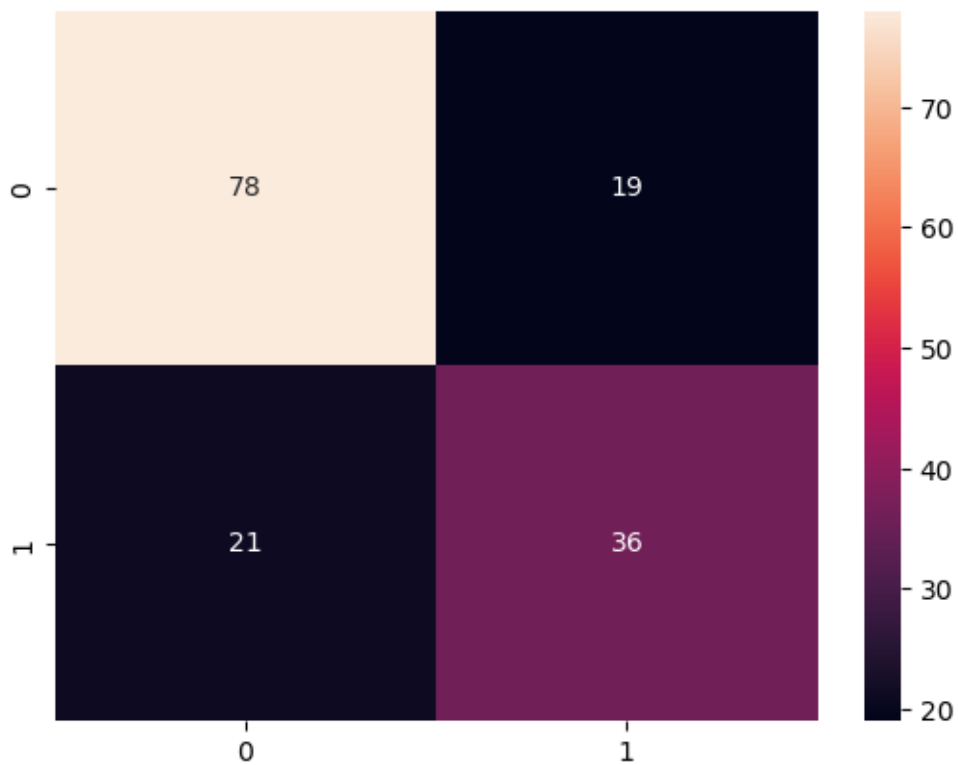
Misclassification Rate of Naive Bayes: 25.97402597402597

```
[86]: 75.97402597402598+24.025974025974026
```

```
[86]: 100.0
```

```
[87]: sns.heatmap(confusion_matrix(y_test,nb_pred),annot=True,fmt="d")
```

```
[87]: <AxesSubplot:>
```



0.19 Classification report of “Naive Bayes”

```
[88]: # classification report of Naive Bayes
print(' Classification Report of Naive Bayes: \n',
      classification_report(y_test,nb_pred,digits=4))
```

Classification Report of Naive Bayes:

	precision	recall	f1-score	support
0	0.7879	0.8041	0.7959	97
1	0.6545	0.6316	0.6429	57

accuracy			0.7403	154
macro avg	0.7212	0.7179	0.7194	154
weighted avg	0.7385	0.7403	0.7393	154

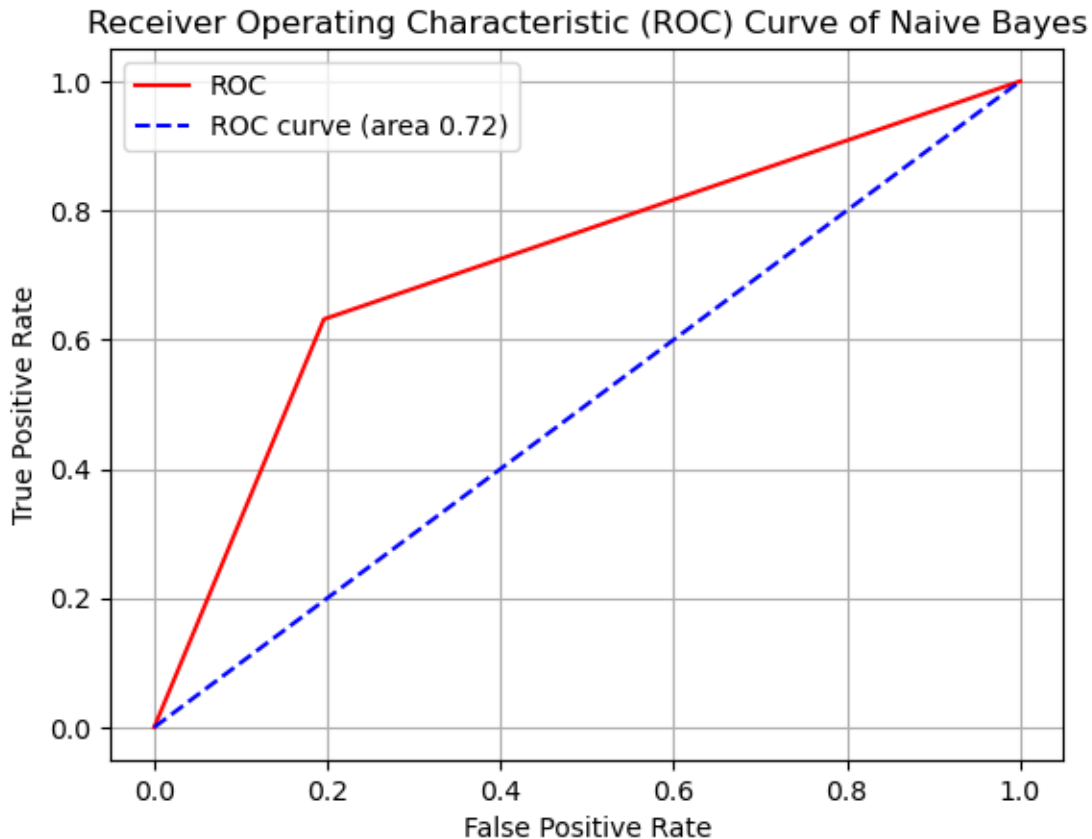
0.20 Roc AUC Score of Naive Bayes

```
[89]: # Area Under Curve

auc = roc_auc_score(y_test, nb_pred)
print("ROC AUC SCORE of Naive Bayes is", auc)
```

ROC AUC SCORE of Naive Bayes is 0.7178513293543136

```
[90]: fpr, tpr, thresholds = roc_curve (y_test, nb_pred)
plt.plot(fpr, tpr, color='red', label='ROC')
plt.plot([0, 1], [0, 1], color='blue', linestyle='--', label='ROC curve (area_
↳ %0.2f)' % auc)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve of Naive Bayes')
plt.legend()
plt.grid()
plt.show()
```



0.21 Confusion matrix of “SVM”

```
[91]: # making the confusion matrix of SVM
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
cm = confusion_matrix(y_test, sv_pred)

print('TN - True Negative {}'.format(cm[0,0]))
print('FP - False Positive {}'.format(cm[0,1]))
print('FN - False Negative {}'.format(cm[1,0]))
print('TP - True Positive {}'.format(cm[1,1]))
print('Accuracy Rate: {}'.format(np.divide(np.sum([cm[0,0], cm[1,1]]), np.
    ↳sum(cm))*100))
print('Misclassification Rate of SVM: {}'.format(np.divide(np.
    ↳sum([cm[0,1], cm[1,0]]), np.sum(cm))*100))
```

TN - True Negative 91
 FP - False Positive 6
 FN - False Negative 20
 TP - True Positive 37

Accuracy Rate: 83.11688311688312
Misclassification Rate of SVM: 16.883116883116884

0.22 Classification Report of SVM

```
[92]: # classification report of SVM
print(' Classification Report of SVM: \n',
      classification_report(y_test,sv_pred,digits=4))
```

Classification Report of SVM:

	precision	recall	f1-score	support
0	0.8198	0.9381	0.8750	97
1	0.8605	0.6491	0.7400	57
accuracy			0.8312	154
macro avg	0.8401	0.7936	0.8075	154
weighted avg	0.8349	0.8312	0.8250	154

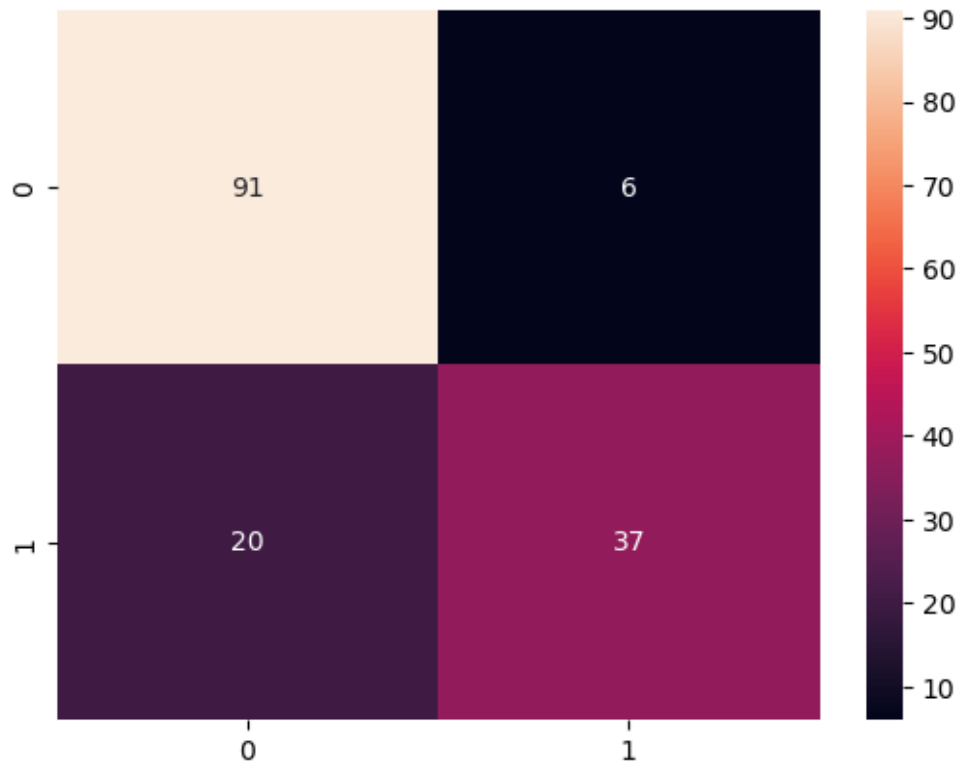
0.23 Roc AUC of SVM

```
[93]: # Area Under Curve
from sklearn.metrics import roc_auc_score
auc = round(roc_auc_score(y_test, sv_pred)*100,2)
print("ROC AUC SCORE of SVM is", auc)
```

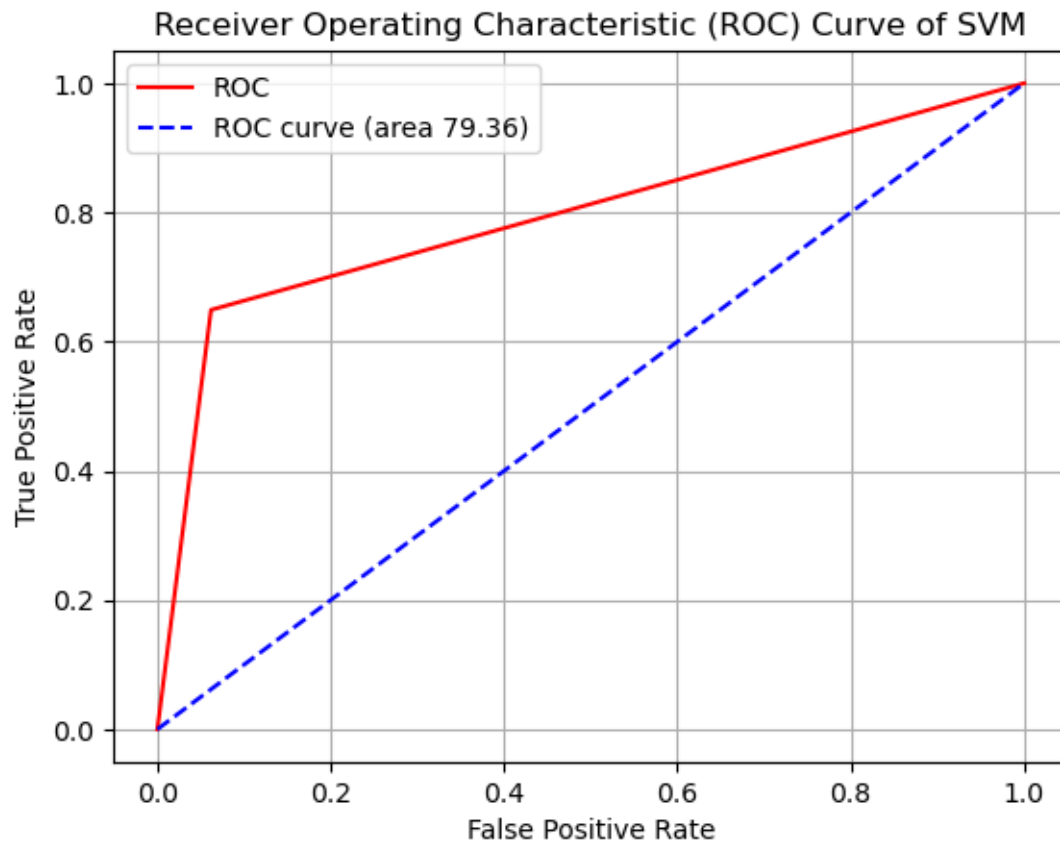
ROC AUC SCORE of SVM is 79.36

```
[94]: sns.heatmap(confusion_matrix(y_test,sv_pred),annot=True,fmt="d")
```

```
[94]: <AxesSubplot:>
```

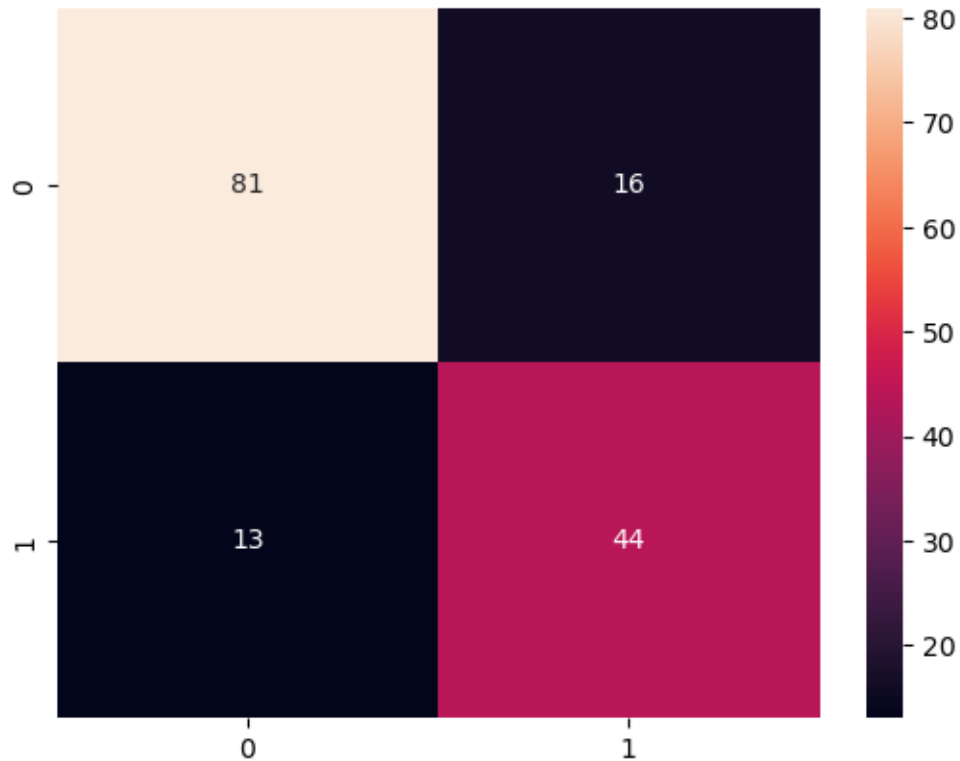
```
[95]: fpr, tpr, thresholds = roc_curve(y_test, sv_pred)
plt.plot(fpr, tpr, color='red', label='ROC')
plt.plot([0, 1], [0, 1], color='blue', linestyle='--', label='ROC curve (area_
    ↪ %0.2f)' % auc)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve of SVM')
plt.legend()
plt.grid()
plt.show()
```



0.23.1 Confusion Matrix of “Decision Tree”

```
[96]: sns.heatmap(confusion_matrix(y_test,dt_pred),annot=True,fmt="d")
```

```
[96]: <AxesSubplot:>
```



```
[97]: # making the confusion matrix of DT
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
cm = confusion_matrix(y_test, dt_pred)

print('TN - True Negative {}'.format(cm[0,0]))
print('FP - False Positive {}'.format(cm[0,1]))
print('FN - False Negative {}'.format(cm[1,0]))
print('TP - True Positive {}'.format(cm[1,1]))
print('Accuracy Rate: {}'.format(np.divide(np.sum([cm[0,0], cm[1,1]]), np.
    ↳sum(cm))))
print('Misclassification Rate of DT: {}'.format(np.divide(np.
    ↳sum([cm[0,1], cm[1,0]]), np.sum(cm))))
```

```
TN - True Negative 81
FP - False Positive 16
FN - False Negative 13
TP - True Positive 44
Accuracy Rate: 0.8116883116883117
Misclassification Rate of DT: 0.18831168831168832
```

```
[98]: # classification report of DT
print(' Classification Report of Decision Tree: \n',
      ↪classification_report(y_test,dt_pred,digits=4))
```

```
Classification Report of Decision Tree:
              precision    recall  f1-score   support

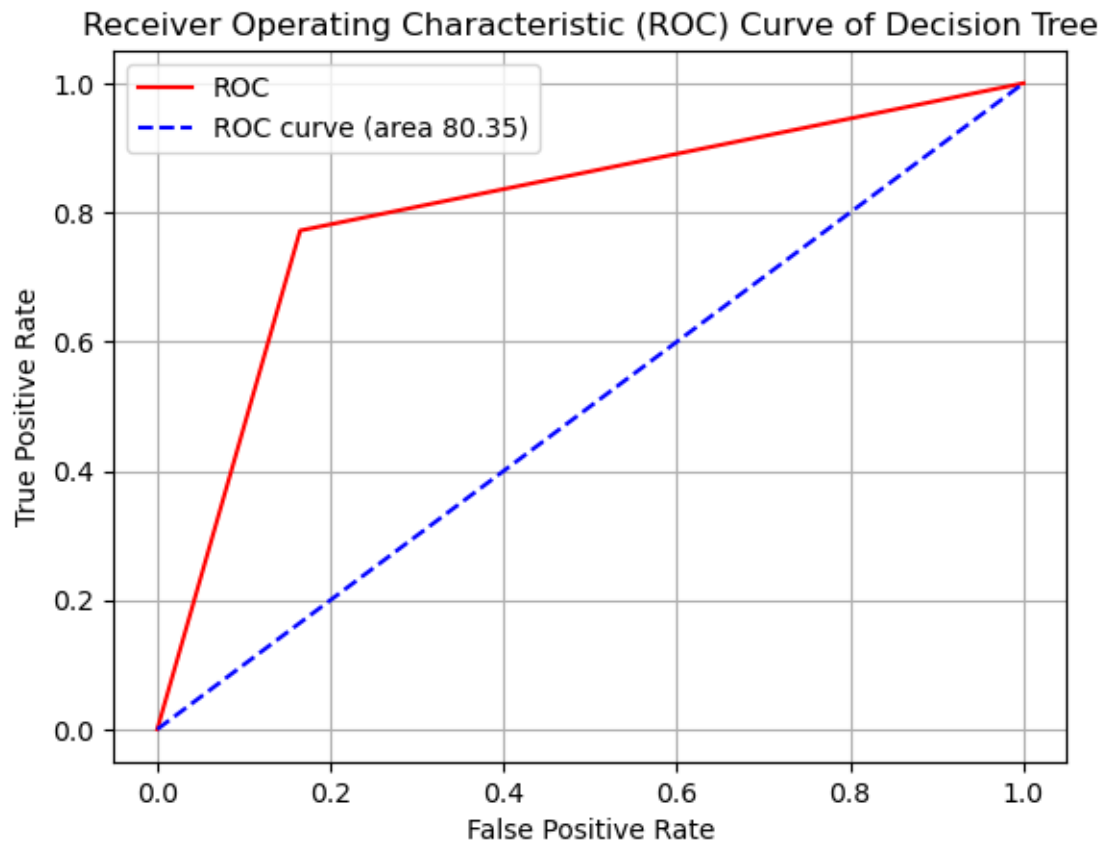
     0       0.8617       0.8351       0.8482         97
     1       0.7333       0.7719       0.7521         57

 accuracy                   0.8117         154
 macro avg       0.7975       0.8035       0.8002         154
 weighted avg    0.8142       0.8117       0.8126         154
```

```
[99]: # Area Under Curve
from sklearn.metrics import roc_auc_score
auc = round(roc_auc_score(y_test, dt_pred)*100,2)
print("ROC AUC SCORE of DT is", auc)
```

ROC AUC SCORE of DT is 80.35

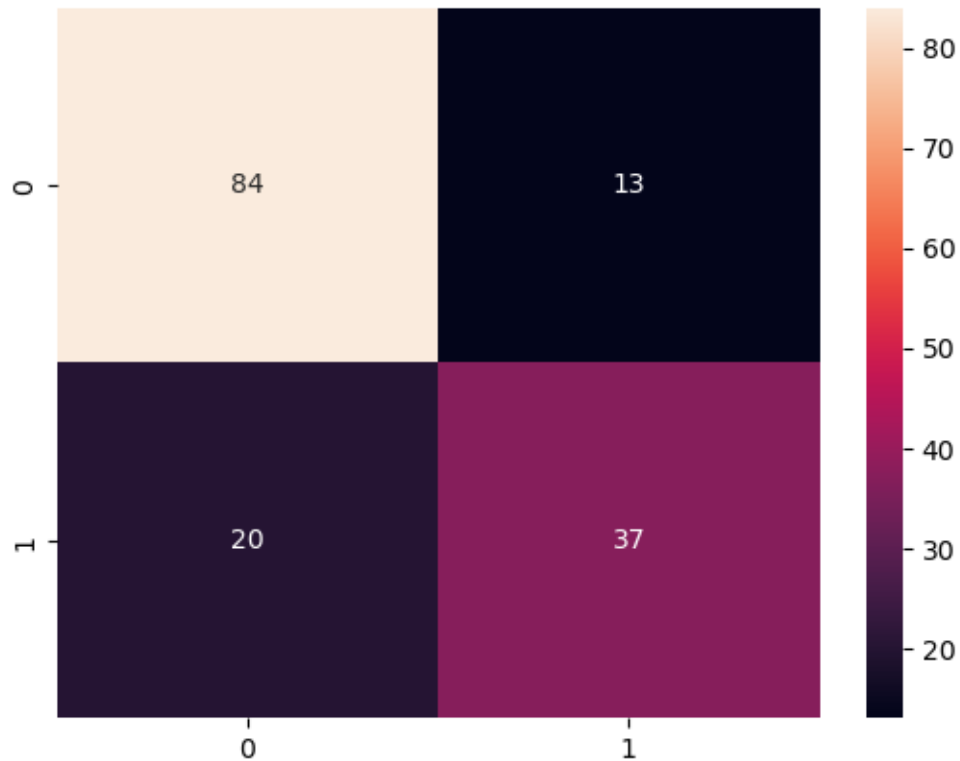
```
[100]: fpr, tpr, thresholds = roc_curve (y_test, dt_pred)
plt.plot(fpr, tpr, color='red', label='ROC')
plt.plot([0, 1], [0, 1], color='blue', linestyle='--', label='ROC curve (area_
      ↪%0.2f)' % auc)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve of Decision Tree')
plt.legend()
plt.grid()
plt.show()
```



0.24 RANDOM FOREST

```
[101]: # heat map of random forest
sns.heatmap(confusion_matrix(y_test, rf_pred), annot=True, fmt="d")
```

```
[101]: <AxesSubplot:>
```



```
[102]: # making the confusion matrix of Random Forest
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
cm = confusion_matrix(y_test, rf_pred)

print('TN - True Negative {}'.format(cm[0,0]))
print('FP - False Positive {}'.format(cm[0,1]))
print('FN - False Negative {}'.format(cm[1,0]))
print('TP - True Positive {}'.format(cm[1,1]))
print('Accuracy Rate: {}'.format(np.divide(np.sum([cm[0,0], cm[1,1]]), np.
    ↳sum(cm))))
print('Misclassification Rate of Random Forest: {}'.format(np.divide(np.
    ↳sum([cm[0,1], cm[1,0]]), np.sum(cm))))
```

```
TN - True Negative 84
FP - False Positive 13
FN - False Negative 20
TP - True Positive 37
Accuracy Rate: 0.7857142857142857
Misclassification Rate of Random Forest: 0.21428571428571427
```

0.25 Classification report of Random Forest

```
[103]: # classification report of Random forest
print(' Classification Report of Random Forest: \n',
      ↪classification_report(y_test,rf_pred,digits=4))
```

Classification Report of Random Forest:

	precision	recall	f1-score	support
0	0.8077	0.8660	0.8358	97
1	0.7400	0.6491	0.6916	57
accuracy			0.7857	154
macro avg	0.7738	0.7576	0.7637	154
weighted avg	0.7826	0.7857	0.7824	154

0.26 Roc AUC of Random Forest

```
[104]: # Area Under Curve
from sklearn.metrics import roc_auc_score
auc = round(roc_auc_score(y_test, rf_pred)*100,2)
print("ROC AUC SCORE of Random Forest is", auc)
```

ROC AUC SCORE of Random Forest is 75.76

```
[105]: fpr, tpr, thresholds = roc_curve (y_test, rf_pred)
plt.plot(fpr, tpr, color='red', label='ROC')
plt.plot([0, 1], [0, 1], color='blue', linestyle='--', label='ROC curve (area_
      ↪0.2f)' % auc)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve of Random Forest')
plt.legend()
plt.grid()
plt.show()
```

